ASSIGNMENT-13.3

Name: E.Ramya

Ht.no: 2303A51282

Task 1: Refactoring – Removing Code Duplication

Objective

To eliminate repeated logic by extracting reusable functions.

Task Description

Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles.Starter (Legacy) Code

Prompt:

Refactor the Python code by removing repeated lines. Create one function to find the area and perimeter of a rectangle using length and width.

Refactor the Python code by removing repeated lines.
Create one function to find the area and perimeter of a rectangle using length and width.

Code:

```python
def rectangle_calculations(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)

rectangle_calculations(5, 10)
rectangle_calculations(7, 12)
rectangle_calculations(10, 15)
```

Output:

```python
def rectangle_calculations(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)

rectangle_calculations(5, 10)
rectangle_calculations(7, 12)
rectangle_calculations(10, 15)
```

Explanation:

- A function is created to calculate the **area and perimeter of a rectangle**.
- The function takes **length and width** as inputs.

- A **docstring** is added to explain what the function does.
- Inside the function, the **area and perimeter are calculated**.
- The results are **printed**.
- The function is **called three times** with different rectangle values.
- This removes **repeated code** and makes the program **clean and easy to reuse**.

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized structures.

Task Description

Use AI to analyze and refactor a script that checks the presence of elements using nested loops.

Prompt:

Refactor the following Python code to improve performance.Remove the inefficient nested loops used to check if names exist in the list.Use a faster method like a set lookup or list comprehension.

Refactor the following Python code to improve performance.

Remove the inefficient nested loops used to check if names exist in the list. Use a faster method like a set lookup or list comprehension.

Code:

```python
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
names_set = set(names)
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

Output:

```
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/
python3.11.exe "c:/AI Assistant Coding/refactor.py"
Charlie is in the list
Eve is not in the list
Bob is in the list
PS C:\AI Assistant Coding>
0 Indexing completed.   Ln 8, Col 41   Spaces: 4   UTF-8   { }
```

Explanation:

- A list called **names** stores some names: Alice, Bob, Charlie, and David.
- Another list called **search_names** contains the names we want to check: Charlie, Eve, and Bob.

- The **outer loop** goes through each name in search_names.
- A variable **found** is set to False at the beginning. This means the name is not found yet.
- The **inner loop** checks each name in the names list.
- If the searched name (s) matches a name in the list (n), **found becomes True**.
- After checking all names:
- If **found is True**, the program prints "name is in the list".
- If **found is False**, the program prints "name is not in the list".
- This process repeats for every name in **search_names**.
- The code works, but it is **slow because it uses nested loops** (loop inside another loop).

Task 3: Refactoring – Extracting Reusable Functions

Objective

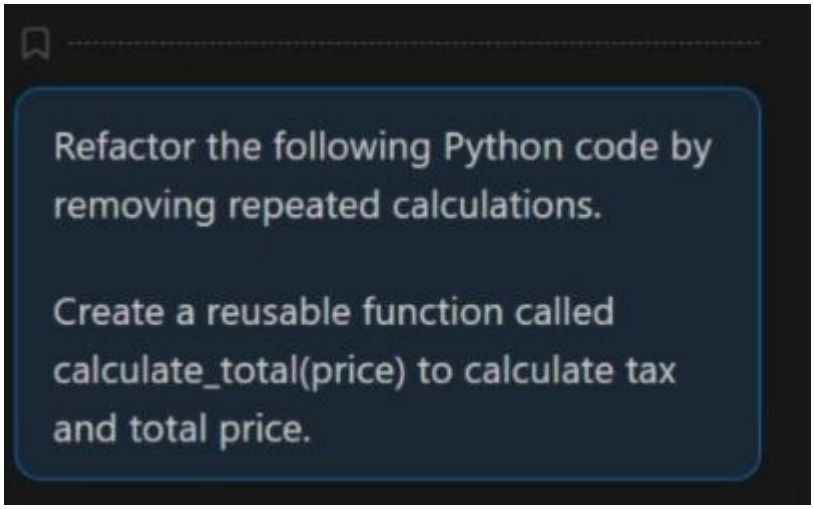
To modularize code by extracting calculations into reusable functions.

Task Description

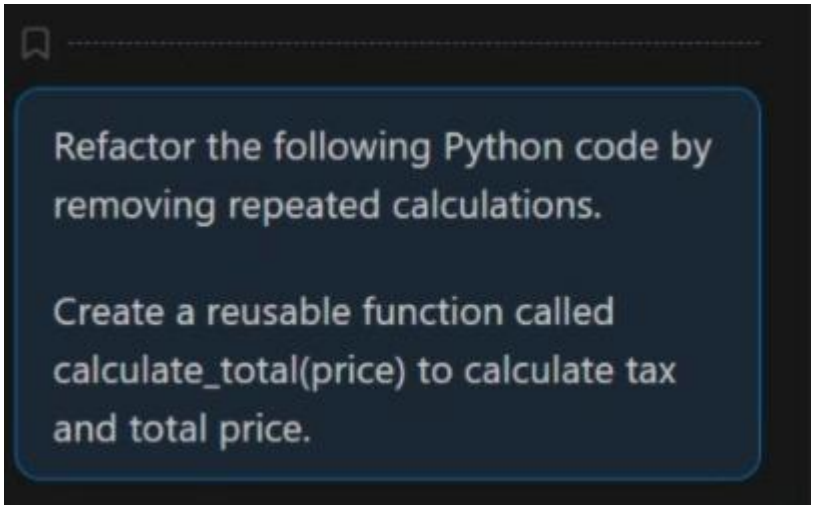
Refactor a legacy script where price and tax calculations are written inline.

Prompt:

Refactor the following Python code by removing repeated calculations.Create a reusable function called calculate_total(price) to calculate tax and total price.



Code:



Output:

```
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/
python3.11.exe "c:/AI Assistant Coding/refactor.py"
Total Price: 295.0
Total Price: 590.0
PS C:\AI Assistant Coding>
```

0 Indexing completed. Ln 7, Col 21 Spaces: 4 UTF-8

Explanation:

- The code calculates the **total price of a product including tax**.
- First, a **price value** is given (250).
- The **tax is calculated as 18% of the price**.
- The **total price** is found by adding the price and tax.
- The program then **prints the total price**.
- The same steps are repeated again for another price value (500).
- This code has **repeated calculations**, so it can be improved by creating a **function to reuse the calculation**.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective

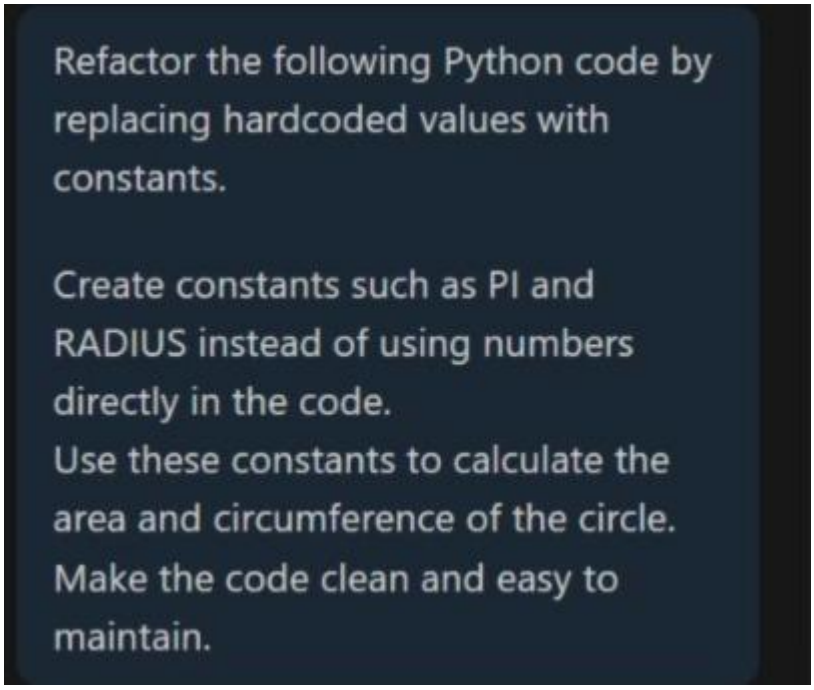
To improve maintainability by replacing magic numbers with named constants.

Task Description

Use AI to identify hardcoded values and replace them with constants.

Prompt:

Refactor the following Python code by replacing hardcoded values with constants.Create constants such as PI and RADIUS instead of using numbers directly in the code.Use these constants to calculate the area and circumference of the circle.



Code:

```
PI = 3.14159
RADIUS = 7

print("Area of Circle:", PI * (RADIUS ** 2))
print("Circumference of Circle:", 2 * PI * RADIUS)
```

Output:

```
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/
python3.11.exe "c:/AI Assistant Coding/refactor.py"
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\AI Assistant Coding>
0 0  Indexing completed.                Ln 5, Col 51   Spaces: 4   UTF-8
```

Explanation:

- The code calculates the **area and circumference of a circle**.
- The value **3.14159** is used as **pi (π)**.
- The value **7** is used as the **radius of the circle**.
- The first line calculates the **area of the circle**.
- The second line calculates the **circumference of the circle**.
- The program then **prints the results**.
- The numbers **3.14159 and 7 are hardcoded values**.
- These values should be replaced with **constants like PI and RADIUS** to make the code cleaner and easier to maintain.

Task 5: Refactoring – Improving Variable Naming and Readability

Objective

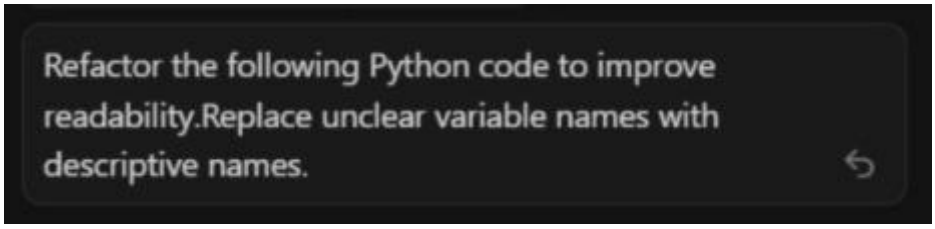
To enhance readability using descriptive variable names and comments.

Task Description

Refactor a script with unclear variable names.

Prompt:

Refactor the following Python code to improve readability. Replace unclear variable names with descriptive names.



Code:

```
width = 10
height = 20
area = width * height / 2
print(area)
```

Output:

Explanation:

- The code assigns **10 to variable a** and **20 to variable b**.

- It calculates the value of **c** by multiplying a and b, then dividing by **2**.
- The calculation follows the formula:
- area = width * height / 2
- The result is stored in variable **c**.
- Finally, the program **prints the value of c**.
- The variable names **a, b, and c are not descriptive**, so they should be replaced with **clear names** to improve readability.